

# 정규화와 모델 선택: Lasso/Ridge · 교차검증 · train/val/test

6.1 정규화와 L1/L2 페널티 · 6.2 교차검증으로  $\lambda$ 를 데이터로 정한다 · 6.3 3분할 원칙 실전

Machine Learning 1 · 6주차

한국과학영재학교 (KSA)

Week 6

## 이번 주차 개요

1996년 티브시라니(Tibshirani)의 한 생각 — 손실함수에 가중치 절댓값의 합을 페널티로 더하면, 무관한 특징의 가중치가 **정확히 0**이 된다. 특징이 수천 개여도 사람이 일일이 고를 필요가 없다.

### 이 주차를 마치면

- “가중치가 작다 = 모델이 단순하다 = 분산이 작다”의 연결고리를 설명하고, L2는 매끄러운 축소(shrinkage)를, L1은 **정확히 0**으로 특징 선택까지 해내는 이유를 짚음
- $\lambda$ 를 편향-분산 트레이드오프 위의 **손잡이**로 해석하고, 계수 경로(coefficient path) 실험으로 확인
- k-겹 교차검증으로  $\lambda$  후보를 검증 MSE로 비교하고, U자형 곡선의 좌측 = 과적합 · 우측 = 과소적합으로 읽음
- train/val/test 3분할 원칙을 적용하고, “정보 흐름의 역주행” (leakage)이 있는 코드를 진단

이미 써본 그 손잡이: Week 5 소프트 마진 SVM의 목적함수  $\frac{1}{2}\|w\|^2 + C \sum_i \xi^{(i)}$ 에서  $\|w\|^2$  자체가 L2 페널티,  $C$ 는 이 장의  $\lambda$ 와 같은 종류의 손잡이었다.

### 세 차시 한 줄

- **6.1** 페널티의 “모양”이 결과를 바꾼다 — 원(L2) 대 마름모(L1), soft-thresholding
- **6.2** 학습 손실이 아니라 **검증** 손실로  $\lambda$ 를 고른다: k-fold CV · 1SE 규칙 · 최적화 편향
- **6.3** train은 배운다, val은 고르고, test는 **딱 한 번** — 정보 흐름에 방벽을 세운다

---

## 6.1 정규화: 편향-분산 복습과 L1/L2 페널티

---

# 손실함수에 페널티 항을 더한다

정규화는 손실함수에 페널티 항을 더해 과적합을 막는다

$$J(w) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2}_{\text{원래 손실(적합도)}} + \underbrace{\lambda R(w)}_{\text{정규화 항(단순함을 요구)}}$$

# 왜 “가중치가 작다”가 “모델이 단순하다”인가

## 핵심 메커니즘

특징을 평균 0·표준편차 1로 표준화해 두면,  $x_j$ 가  $\Delta$ 만큼 변할 때 예측치는  $|w_j| \Delta$  이상으로 변할 수 있다 —  $|w_j|$ 는 “특징  $x_j$ 가 출력을 흔들어놓을 수 있는 **강도**”다.

모든  $|w_j|$ 가 작으면 입력이 아무리 요동쳐도 출력은 거의 못 흔들린다. 4.1에서 “노이즈 한 점에 예측이 크게 출렁이는 모델” = 분산이 큰 모델이었는데, 가중치가 작은 모델은 그 **정 반대**(흔들림이 억눌림) → 분산이 작다.

- “가중치 축소(shrinkage) = 유효 복잡도 감소”가 정규화의 핵심.



Robert Tibshirani (Stanford) — 1996년 논문 “Regression Shrinkage and Selection via the Lasso”로 L1 정규화(Lasso)를 제안

# 페널티의 모양이 다르면 결과도 다르다

## L2 정규화 (Ridge)

$$R(w) = \|w\|_2^2 = \sum_j w_j^2$$

모든 가중치를 **매끄럽게** 0 쪽으로 축소(shrinkage). 좀처럼 정확히 0이 되지는 않는다.

## L1 정규화 (Lasso)

$$R(w) = \|w\|_1 = \sum_j |w_j|$$

중요하지 않은 가중치를 **정확히 0**으로 만들어버리는 경향. 그래서 정규화 + **특징 선택**을 자동 수행.

둘 다 “가중치를 작게 만든다”는 목표는 같지만, **모양이 다르면 결과도 다르다**.

# L1은 왜 0으로 “뭉개지는”가 — soft-thresholding

- L1은 0에서 뽕족
- 원래 손실이  $w_j$ 를 0에서 밖으로 당기려는 힘이  $\lambda$ 보다 약하면  $w_j$ 는 0에 박혀 나오지 않는다.

## soft-thresholding 공식과 좌표하강

$$w_j \leftarrow \mathcal{T}_\lambda(z_j) = \text{sign}(z_j) \max(|z_j| - \lambda, 0)$$

| 조건                 | 결과                               |
|--------------------|----------------------------------|
| $ z  \leq \lambda$ | <b>0</b> (0으로 스냅)                |
| $z > \lambda$      | $z - \lambda$ ( $\lambda$ 만큼 축소) |
| $z < -\lambda$     | $z + \lambda$                    |

**soft-thresholding:** “신호가 약하면 0으로 스냅, 강하면 초과분만 통과”.

### 왜 좌표하강(coordinate descent)

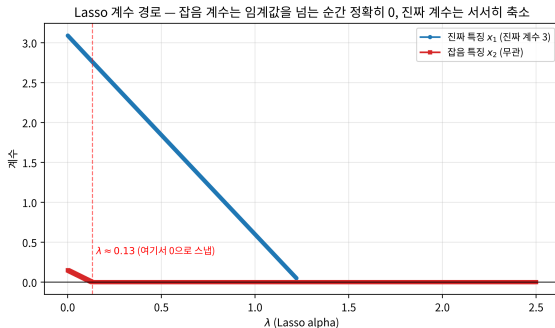
$|w|$ 가 미분불가능해 **구배하강을 그대로 쓸 수 없으므로**, 한 번에  $w_j$  하나만 이 공식으로 갱신하고 나머지는 고정. scikit-learn Lasso가 내부적으로 하는 일이 이 공식의 반복.

# 계수 경로: 0은 점진적이지 않고 “도착지”

## Lasso 계수 경로: $\lambda$ 변화에 따른 기록

**진짜  $x_1$ :** 3.09에서 점점 줄어, 잡음 스텝 ( $\approx 0.13$ )보다 훨씬 늦은  $\lambda \approx 1.2$ 에서야 정확히 0.

**잡음  $x_2$ :** 0.15에서 시작해  $\lambda \approx 0.13$  넘는 순간 정확히 0, 그 뒤 한 번도 0에서 벗어나지 않음.



## 기하학적 직관: 원 vs 마름모

- **L2**: 제약 영역은 원(구) — 등고선이 표면 어디든 만남.
- **L1** 제약 영역은 마름모

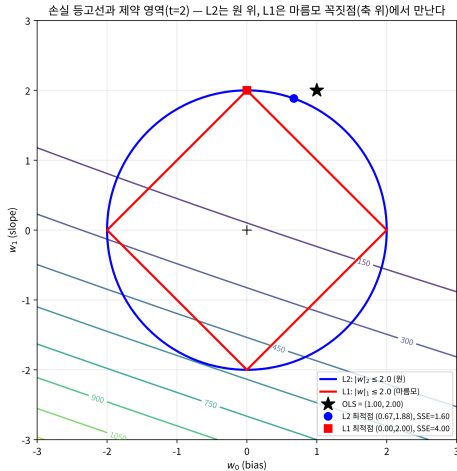
**2차원 숫자:**  $X = [1, 2, 3, 4]$ ,  $y = [3, 5, 7, 9]$ (직선  $y = 2x + 1$ , OLS 해  $(1, 2)$ )에서 제약  $t = 2$ :

|         | 최적점                             | SSE            |
|---------|---------------------------------|----------------|
| L2(원)   | $(0.67, 1.88)$ — 둘 다 0 아님       | $\approx 1.60$ |
| L1(마름모) | $(0, 2.0)$ — $w_0$ 축 <b>꼭짓점</b> | $\approx 4.00$ |

### 차원이 클수록 희소해진다

2차원에선 꼭짓점이 4개뿐이지만 특징  $p$ 개면  $p$ 차원 정다면체로 변해 꼭짓점(=어떤 좌표가 0인 점)이 **방대해짐**. 특징이 많을수록 “등고선이 꼭짓점에서 만남” 확률이 커지므로, Lasso의 희소(sparse)한 해는 **차원이 클수록 더 흔해진다**.

# L1 최적점은 꼭짓점에 닿는다



L1은 마름모 꼭짓점에서 만난다

# Ridge 경사하강 구현

```
def ridge_gradient_descent(X, y, lam, alpha, epochs):
    m, n = len(X), len(X[0])
    w = [0.0] * (n + 1) # w[0] = bias
    for _ in range(epochs):
        grad = [0.0] * (n + 1)
        for i in range(m):
            pred = w[0] + sum(w[j+1] * X[i][j] for j in range(n))
            error = pred - y[i]
            grad[0] += error
            for j in range(n):
                grad[j+1] += error * X[i][j]
        for j in range(n + 1):
            # bias(w[0])는 관례적으로정규화에서제외
            reg = lam * w[j] if j > 0 else 0
            w[j] -= alpha * (grad[j] / m + reg)
    return w
```

$\lambda$  증가 시 가중치 0에 가까워짐

## 손으로 한 번: Ridge 정규방정식

$$w^* = (X^T X + \lambda D)^{-1} X^T y \quad (D \text{는 bias 제외 대각행렬})$$

$X = [1, 2, 3], y = [3, 5, 7]$  (정규화 없는 해  $w^* = [1, 2]$ )에  $\lambda = 2$ :

$$X^T X + \lambda D = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 16 \end{bmatrix}$$

행렬식  $3 \times 16 - 6 \times 6 = 12$ ,  $X^T y = [15, 34]$ 에 곱하면  $w^* = [3, 1]$ .

## 자주 하는 실수: bias(절편)까지 정규화

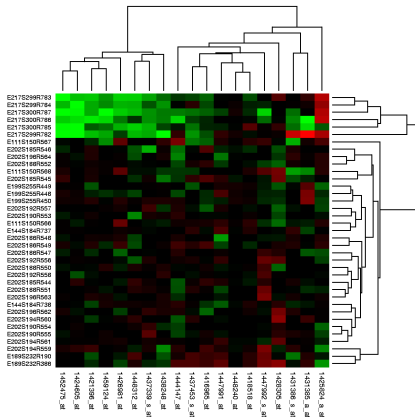
|                     | $\lambda$ | 올바른 $D = \text{diag}(0, 1)$ | 틀린 $D = I$ (bias도 정규화) |
|---------------------|-----------|-----------------------------|------------------------|
| bias 정규화 시 $w^*$ 왜곡 | 2         | [3.00, 1.00]                | [0.82, 1.82]           |
|                     | 10        | [4.33, 0.33]                | [0.57, 1.28]           |

### 왜 안 좋은가

bias는 “모델 복잡함”이 아니라 데이터 전체의 **평균적 높이**. 기울기가 늘리면 bias가 보상하며 더 커져야 한다(3.00→4.33)는데, bias까지 정규화하면  $\lambda$ 가 커질수록 0 쪽으로 반대 방향으로 끌려가(0.82→0.57)  $y$ 의 평균이 원점에서 먼 데이터(주택 가격)에서 체계적으로 아래로 치우친다.

## 유명한 사례: Lasso는 유전체학에서 태어났다

- Lasso는 특징이 많고 신호가 소수인 문제에서 작동
- Lasso가 1차 후보인 이유: soft-thresholding



유전자 발현(gene expression) 데이터 히트맵 — 각 행은 유전자, 각 열은 샘플. 1996년 Tibshirani가 Lasso를 검증한 것과 같은 종류의 “특징(유전자) 수천 개 중 진짜 신호는 소수” 데이터

## 실습: L1과 L2의 “정확히 0” 차이

```
from sklearn.linear_model import Ridge, Lasso
x1 = np.array([1,2,3,4,5,6], dtype=float)
x2 = np.array([5,3,8,1,9,2], dtype=float) # 와y 무관한잡음
y = 2 * x1 + 1
X = np.column_stack([x1, x2])
for alpha in [0.1, 1.0, 5.0]:
    ridge = Ridge(alpha=alpha).fit(X, y)
    lasso = Lasso(alpha=alpha).fit(X, y)
    print(f"alpha={alpha}: ridge x2={ridge.coef_[1]:.4f}, "
          f"lasso x2={lasso.coef_[1]:.4f}")
```

**Ridge:**  $x_2$  계수는 점점 작아지지만 **정확히 0이 아님**  
(-0.0004  $\rightarrow$  -0.0040  $\rightarrow$  -0.0153)

**Lasso:** 세  $\alpha$  모두  $x_2$  계수를 **정확히 0.0000**으로 —  
잡음 특징을 아예 “제거”

실전 데이터에서 Lasso가 이런 특징들의 계수를 0으로 만들어주는 것이 곧 “특징 선택을 자동으로 해낸다”의 실제 의미.

## 한 뼛 더: 등가 특징 두 개면 어느 쪽을 남기는가

$x_1$ 과 완전히 같은 정보를 담는 등가 특징  $x_{2c} = 3x_1 - 4$ (완전 상관)를 추가 + 잡음  $x_3$ :

```
x2c = 3 * x1 - 4.0          # 과x1 등가완전( 상관) 특징
x3  = np.array([2,9,1,7,3,6], dtype=float)
Xc  = np.column_stack([x1, x2c, x3])
print(Lasso(alpha=2.0).fit(Xc, y).coef_)
# [0. 0.5905 0.] 순서 (: [x1, x2c, x3])
```

**Lasso는 등가 특징 중 하나만 남김**

**Ridge는 가중치 분산, Lasso는 희소 해**

## $\lambda$ 는 편향-분산 트레이드오프 위의 “손잡이”

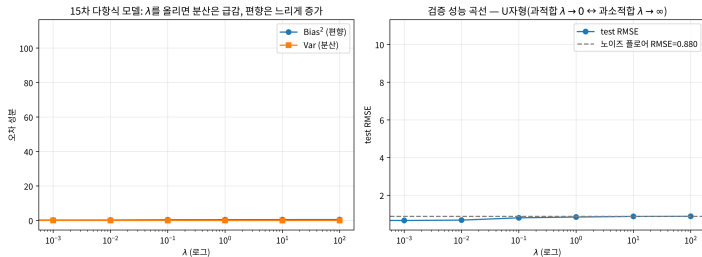
$$\mathbb{E} \left[ (y - \hat{f}(x))^2 \right] = \underbrace{\left( \text{Bias}[\hat{f}(x)] \right)^2}_{\text{구조적으로 못 맞추는 정도}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{훈련이 바뀔 때 예측이 흔들림}} + \underbrace{\sigma^2}_{\text{데이터의 잡음}}$$

## 숫자로 확인: $\lambda$ 축을 훑는다

| $\lambda$ | Bias <sup>2</sup> | Var    | test RMSE |
|-----------|-------------------|--------|-----------|
| 0         | 5.64              | 110.90 | 10.81     |
| 0.001     | 0.24              | 0.030  | 0.66      |
| 0.01      | 0.26              | 0.015  | 0.68      |
| 0.1       | 0.40              | 0.008  | 0.80      |
| 1.0       | 0.44              | 0.004  | 0.85      |
| 10        | 0.48              | 0.004  | 0.88      |
| 100       | 0.49              | 0.004  | 0.89      |

- $\lambda = 0.001$ : **RMSE 0.66** 곡선의 바닥

# U자형 곡선과 잡음 플로어



## Bias<sup>2</sup>와 Var, U자형 곡선과 잡음 플로어

### U자형 곡선

왼쪽 날개 = 과적합(Var 지배), 오른쪽 날개 = 과소적합(Bias<sup>2</sup> 지배), 바닥 = 최적  $\lambda$ 의 대략적 위치.

## 자주 묻는 질문 (1)

### Q. 항상 Lasso(L1)를 쓰는 게 낫나요?

- 아님. 특징이 강하게 상관되면(집 크기 대 방 개수) Lasso는 하나만 임의로 남기고 나머지를 0으로 — **불안정**(데이터 살짝 바뀌도 남는 특징이 달라질 수 있음).
- Ridge는 상관 특징들의 가중치를 비슷하게 나눠 갖는 경향 → **안정적**.
- 실전: 둘을 섞은 **엘라스틱넷**(Zou·Hastie 2005),  $R(w) = \alpha\|w\|_1 + (1 - \alpha)\|w\|_2^2$ .

### Q. Lasso가 0으로 만든 특징이 사실 중요했지만 효과 작아 떨어진 경우, 어떻게 구별?

- 구분할 수 없는 것이 정답에 가까움 —  $w_j = 0$ 은 “무관하다”는 증명이 아니라 그  $\lambda$ 에서 최적화가 내린 **선택의 결과**.
- 진짜 계수 3이면  $\lambda = 0.13$  넘겨도 살아남지만 0.2였으면 같은  $\lambda$ 에서 0 — 신뢰도는 “진짜 계수 > 임계  $\lambda$ ” 전제에 달려 있고, 그 전제는 데이터 이전에 보장되지 않음.
- 줄이는 법: 특징 **스케일링** +  $\lambda$ 를 6.2의 CV로 “너무 크게” 잡지 않게.

## 자주 묻는 질문 (2)

### Q. $\lambda$ 가 너무 크면?

- $\lambda \rightarrow \infty$ 이면 정규화 항이 원래 손실을 압도  $\rightarrow$  모든 가중치(bias 제외)가 0에 수렴, “항상 평균값만 예측하는” 가장 단순한 모델로 수렴(분산  $\rightarrow 0$ , 편향 극도로 큼 = 과소적합 극단).

### Q. $|w|$ 가 미분불가능하면 Lasso는 최적화할 수 없는 거 아닌가요?

- 아님 — 0에서의 서브그래디언트( $[-1, 1]$ )로 최적조건을 정의하면 해가 존재하고 **좌표하강**이 효율적으로 찾음.
- L2는 모든 점에서 미분가능  $\rightarrow$  구배하강·달힌형 모두 가능. Lasso는 0에서 뽀족  $\rightarrow$  전용 알고리즘(좌표하강·내적점). 실전에서 Lasso를 그냥 `.fit()`하는 것은 이 전용 알고리즘을 쓰는 것.

### Q. 정규화 계수 $\lambda/2$ vs $\lambda$ , sklearn alpha?

- 기호가 서로 다름( $\frac{\lambda}{2m}\|w\|^2$ ,  $\frac{\lambda}{2}\|w\|^2$ ,  $\lambda \sum w_j^2$ ). sklearn Ridge(alpha=a)는  $\frac{a}{2} \sum w_j^2$ .
- **상대적 강도만** 의미 있고 절대값은 무의미  $\rightarrow$  다른 라이브러리의  $\lambda$ 를 숫자로 비교하지 말 것.  
“적절한 강도”는 6.2의 CV로 **데이터가** 정한다.

## 확인 문제 6.1

- ① 참인 것을 모두 고르라: (a) Ridge에서  $\lambda \rightarrow 0$ 이면 해는 OLS로 수렴. (b) Lasso에서  $\lambda \rightarrow \infty$ 이면 bias 제외 모든 계수 정확히 0. (c) Lasso는 항상  $y$ 와 상관 큰 특징만 남김. (d) L1/L2의 “적당한”  $\lambda$ 는 train loss가 가장 작은 곳에서 고른다.

힌트: (c)는 등가 특징 예, (d)는 6.2 서두 “train loss만 보면 항상  $\lambda = 0$ ”

- ② **soft-thresholding 손계산.**  $f(w) = \frac{1}{2}(2 - w)^2 + \lambda|w|$ 에서  $\lambda = 0.5$ 와 3일 때 최적 해를  $\mathcal{T}_\lambda(z)$  ( $z = 2$ )로 구하고, “왜 0이 되고/안 되는지”  $|z| \leq \lambda$  조건으로 설명.
- ③ **서브그래디언트.**  $|w|$ 의  $w_0 > 0, w_0 < 0, w_0 = 0$ 에서 서브그래디언트( $1, -1, [-1, 1]$ )를 구하고,  $f(w) = \frac{1}{2}(z - w)^2 + \lambda|w|$ 에 대해  $w^* = 0 \iff |z| \leq \lambda$ 를 증명한 뒤  $|z| > \lambda$ 일 때  $w^* = \text{sign}(z)(|z| - \lambda)$  확인(soft-thresholding의 증명).
- ④ **개념.** 등가 특징 예에서 Lasso가  $x_1$  대신  $x_{2c}$ 를 남긴 이유를 L1 페널티 관점에서 설명(힌트: 같은 예측 만드는  $(w_1, w_{2c})$  중 어느 쪽이  $|w_1| + |w_{2c}|$ 가 작나).

## 6.1 정리

- 정규화는 손실함수에 “가중치 크기” 페널티 한 항을 더해 유효 유연성을 낮춤 — 4.1의 언어로  $\lambda$ 를 올리면 **분산을 편향과 바꾸는 것**. 살짝만 올려도 Var가 4자리수 급감할 만큼 “분산 제거”에 효율적.
- L2는 모든 가중치를 **매끄럽게** 축소, L1은 soft-thresholding( $|z_j| \leq \lambda$ 이면 0)으로 약한 특징을 **정확히 0**에 스냅시켜 특징 선택을 무상으로 — 기하학적으로 “원 위” vs “마름모 꼭짓점 위”.
- bias는 관례적으로 정규화 대상에서 **제외**.

“ $\lambda$  자체는 얼마가 적당한가” — U자형 곡선의 바닥을 정직하게 고르는 절차는 6.2(교차검증)·6.3(train/val/test)에서.

---

## 6.2 교차검증: $\lambda$ 를 데이터로 정한다

---

# 왜 검증 데이터가 필요한가

$\lambda$ 는 학습 전 정하는 하이퍼파라미터

- 검증 데이터(validation set)로 성능을 재야 한다.

데이터가 넉넉하지 않을 때: k-겹 교차검증

**k-겹 교차검증**(k-fold cross-validation, Week 4.2에서 아이디어만 먼저 봤다): 데이터를  $k$ 개로 쪼갬 뒤, 매번 한 조각을 검증용으로 남기고 나머지로 학습하는 것을  $k$ 번 반복해 성능을 평균냄 — 전체를 한 번씩은 검증에도 학습에도 써보는 셈. 이번 절에서는  $\lambda$  선택에 실제로 적용하고, 코드와 CV 추정값 자체의 편향까지 엄밀하게 다룬다.

# k-fold 분할 코드

```
def k_fold_split(data, k, fold_idx):  
    n = len(data)  
    fold_size = n // k  
    start = fold_idx * fold_size  
    end = start + fold_size if fold_idx < k - 1 else n  
    val = data[start:end]  
    train = data[:start] + data[end:]  
    return train, val
```

- fold\_idx번째 블록을 검증용(val), 나머지를 학습용(train) 반환. 마지막 fold는 남은 것을 전부 포함.

## 손으로 한 번: “아무것도 못 배우는” 모델 5-fold CV

| fold | val   | train 평균  | fold별 MSE    |
|------|-------|-----------|--------------|
| 0    | 1, 2  | 6.5       | 25.25        |
| 1    | 3, 4  | 6.0       | 6.50         |
| 2    | 5, 6  | 5.5       | 0.25         |
| 3    | 7, 8  | 5.0       | 6.50         |
| 4    | 9, 10 | 4.5       | 25.25        |
|      |       | <b>평균</b> | <b>12.75</b> |

fold 0 풀어쓰기:  $\text{train} = \{3, \dots, 10\}$ 의 평균  $= (3 + \dots + 10)/8 = 6.5$ ,  $\text{MSE} = [(1 - 6.5)^2 + (2 - 6.5)^2]/2 = [30.25 + 20.25]/2 = 25.25$ .

“정직한” 기준선: 진짜 평균 5.5로 예측했을 때의  $\text{MSE} = \frac{82.5}{10} = 8.25$ .

# CV 추정값은 “진짜 성능”이 아니다

CV 추정값(12.75)이 정직한 값(8.25)보다 53%나 높다

각 fold의 “train 평균”(6.5, 6.0, 5.5, 5.0, 4.5)이 진짜 평균 5.5 주위를 흔들리기 때문. train 평균이 1.0만큼 어긋난 fold 0에서 그 어긋남이 val 오차에 그대로 더쳐진다.

**CV 추정값은 “진짜 성능”이 아니라 “검증 절차의 시뮬레이션”** — 절차 안의 작은 흔들림도 포함되므로 특정 모델 하나엔 오히려 높게(비관적으로) 나올 수도.

- CV 낙관적 편향은 모델 선택 단계에서 발생

## k-fold CV로 최적 $\lambda$ 찾기

```
def cross_validate_lambda(X, y, lambdas, k=5, alpha=0.01, epochs=500):
    data = list(zip(X, y))
    results = {}
    for lam in lambdas:
        val_errors = []
        for fold in range(k):
            train, val = k_fold_split(data, k, fold)
            Xtr, ytr = [d[0] for d in train], [d[1] for d in train]
            Xva, yva = [d[0] for d in val], [d[1] for d in val]
            w = ridge_gradient_descent(Xtr, ytr, lam, alpha, epochs)
            val_errors.append(mse(Xva, yva, w))
        results[lam] = sum(val_errors) / len(val_errors)
    return results
```

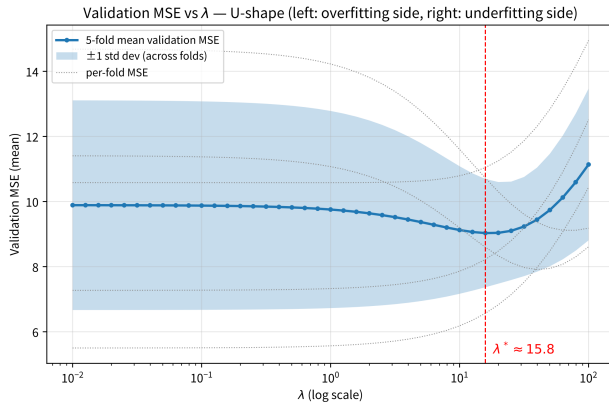
“학습 손실이 아니라 검증 손실로  $\lambda$ 를 고른다”는 원칙을 코드로 확인.

## U자형이 실제로 그려진다

| $\lambda$ | 평균 검증 MSE   |
|-----------|-------------|
| 0.0       | 12.520      |
| 0.5       | 10.872 ← 최적 |
| 2.0       | 11.780      |
| 10.0      | 13.740      |

- 정규화 전혀 없음( $\lambda = 0$ )도, 너무 강함( $\lambda = 10$ )도 검증 MSE가 오히려 나빠지고, 그 사이 어딘가에 최적점.
- 이 U자 모양 = 6.1의 편향-분산 트레이드오프가 실제 숫자로 나타난 것.

# U자형 곡선: 좌측=과적합, 우측=과소적합



$\lambda$ 에 따른 5-fold 평균 검증 MSE

- **최소점:** 둘의 합이 가장 작은 위치.

# 1SE 규칙: 최소점은 뾰족한 꼭짓점이 아니다

- CV 최소점은 추정치
- 같은 생성 과정을 시드 10개로 반복하면 각 시드가 고른  $\lambda^*$ 가 **5.0~20.0까지**(약 4배) 흩어짐  
→ 평탄한 대지 (plateau).

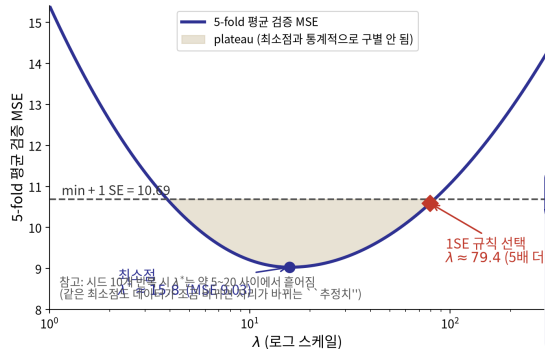
(1-standard-error rule, Tibshirani 1996)

CV MSE가 (최소점 + **fold 표준편차 1개**) 이내인 후보 중 **가장 큰  $\lambda$**  — 즉 **가장 단순한 모델** — 을 고른다.

논리: “CV 평균의 불확실성은  $\pm 1$  표준편차 급이고, 그 범위 안에서는 더 단순한 쪽을 고르는 것이 합리적 tie-breaker”.

# 1SE 규칙 적용: 5배 큰 $\lambda$

1SE 규칙: 평탄한 대지에서 가장 단순한 모델 고르기



- 최소점  $\lambda \approx 15.8$  (MSE 9.03), fold 표준편차 1개(1.66)를 더한 허용 상한 = **10.69**.
- 이 범위 안에 드는 후보 중 가장 큰  $\lambda = \lambda \approx 79.4$  (MSE 10.59) — 최소점보다 **5배** 더 큰  $\lambda$ .

## 실전 의미

더 작고 단순한 모델이 “통계적으로 구별되지 않는” 수준으로 좋다. 6.1의 Lasso로 치면, 정확도를 사실상 그대로 유지하면서 더 많은 특징을 0으로 만들어 **더 희소하고 해석하기 쉬운** 모델을 얻음.

## 손으로 한 번: 4-fold 분할 직접 나눠보기

샘플 12개(인덱스 0~11)를  $k = 4$ 로 나누면  $\text{fold\_size} = 12 // 4 = 3$ . 각  $\text{fold\_idx}$ 에 대해  $\text{start} = \text{fold\_idx} \times 3$ ,  $\text{end} = \text{start} + 3$ (마지막 fold만  $n$ 까지):

| fold_idx | 검증용(val)  | 학습용(train)              |
|----------|-----------|-------------------------|
| 0        | 0, 1, 2   | 3, 4, ..., 11           |
| 1        | 3, 4, 5   | 0, 1, 2, 6, 7, ..., 11  |
| 2        | 6, 7, 8   | 0, 1, ..., 5, 9, 10, 11 |
| 3        | 9, 10, 11 | 0, 1, ..., 8            |

## 자주 하는 실수: 섞지 않은 데이터를 fold로

k\_fold\_split은 **순서 그대로** 앞에서부터 자른다 — 데이터가 정렬되어 있으면 심각한 문제.

| 분할        | fold별 정확도                    | 평균   | fold 간 $\sigma$ |
|-----------|------------------------------|------|-----------------|
| 섞지 않음(순서) | 1.00, 1.00, 0.50, 0.00, 0.00 | 0.50 | 0.447           |
| 섞음(시드 42) | 0.50, 0.67, 0.33, 0.33, 0.67 | 0.50 | 0.149           |

**fold별 흠어짐을 봐야 평균을 읽을 수 있다**

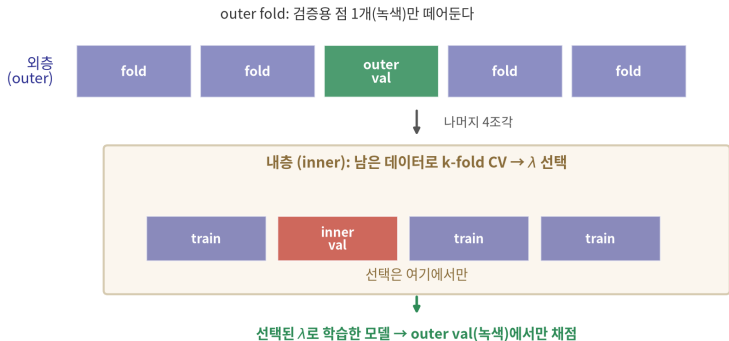
## $\lambda$ 를 고르는 것 자체가 “선택”: 최적화 편향

- ① 각  $\lambda$  후보마다 CV 평균 계산(데이터 사용).
- ② 그중 **가장 작은 값을 가진  $\lambda$** 를 고른다 — 이것도 같은 데이터로 한 **선택**.
- ③ 그 최소점의 값을 “선택된 모델의 기대 성능”으로 보고.

### 선택 편향 구조 동일

후보가 많을수록, 각 후보의 CV 평균 간 흔들림이 클수록, **곡선의 최소점은 평균적으로 낙관적**. CV 곡선 자체가 그 데이터로 “조정된(tuned)” 결과이므로 그 최솟값은 보아온 숫자 중 제일 좋은 숫자.

# nested 교차검증 구조

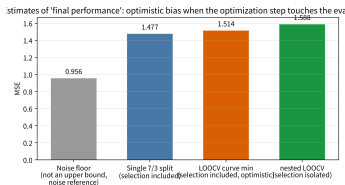


정보 흐름: 내층 → 외층. 외층 검증 데이터는 이 그림 어디의 선택에도 쓰이지 않는다.

## 외층 검증, 내층 $\lambda$ 선택

## 편향이 실제로 얼마인지: 숫자로

| 추정 방법             | 추정 MSE       | 선택이 평가에 닿는가                |
|-------------------|--------------|----------------------------|
| 노이즈 플로어(참고선)      | 0.956        | —                          |
| (a) 단일 7/3 분할     | 1.477        | 평가 3개에는 안 닿음               |
| (b) LOOCV 곡선의 min | <b>1.514</b> | <b>당함</b> — 곡선 전체가 10개로 그림 |
| (c) nested LOOCV  | 1.588        | 안 닿음(선택은 전부 내층 9개)         |



핵심은 (b) vs (c): (c)가 (b)보다 0.07 높게 — 이 갭이 “곡선 최소점을 최종 성능으로 보고한” 것에 배어 있던 낙관적 편향.

## 실전 가이드라인: “항상 nested CV”가 아니다

- 보통 CV로  $\lambda$ 를 **고르는 것(selection)**은 당연 — 그것이 val의 정당한 용도.
- **보고(reporting)**할 때는, 어떤 선택에도 쓰인 적 없는 **홀드아웃 데이터**(6.3의 test set)로 재는 것이 원칙.
- 홀드아웃이 없는 상황(경진대회, 데이터 전부 소진)이라면 nested CV가 “같은 데이터로 정직한 추정치를 얻는” 방법.

### 이 절의 $\lambda$ 선택이 만드는 최적화 편향

“어떤 데이터로 무엇을 고르고, 어떤 데이터로 최종 측정하는가”를 엄격한 원리로 정리하는 것이 바로 6.3의 주제.

## 자주 묻는 질문 (1)

### Q. $k$ 는 몇으로?

- $k = 5$  또는 10이 실전 기본값.
- $k$ 가 클수록( $k = m$ , **LOOCV**) 각 fold 학습 데이터가 거의 전체  $\rightarrow$  편향 줄지만, fold를  $m$ 번 다시 학습  $\rightarrow$  비용  $\uparrow$  · fold 간 분산  $\uparrow$ .
- $k$ 가 작을수록( $k = 3$ )은 빠르지만 fold 학습 데이터 작아 편향  $\uparrow$ .
- 5 ~ 10이 실용적 절충점.

### Q. 교차검증은 항상 해야 하나요?

- 데이터가 아주 많으면(수백만 개) 한 번만 train/val로 나뉘어도 검증셋이 충분히 커 안정적인 평가.
- 적을수록(수백~수천) 한 번의 분할이 우연히 “쉬운/어려운” 검증셋을 뽑을 위험이 커  $\rightarrow$  교차검증의 가치  $\uparrow$ .

## 자주 묻는 질문 (2)

### Q. 경진대회(Kaggle)에서 test 라벨이 안 공개되면?

- 공개 **public leaderboard**는 사실상 val, 비공개 **private**는 test 역할.
- public 점수로  $\lambda$ 를 골라 계속 제출하면, public을 “20번 봤다”는 뜻  $\rightarrow$  § 최적화 편향 관점에서 public 점수는 **부풀려진** 추정치. private의 제출 횟수 제한이 바로 이 선택 편향을 통제하기 위함.
- **교훈: 같은 검증 조각으로 비교한 후보가  $m$ 개면, 그 검증 점수는  $m$ 개 중 max를 본 셈.**

### Q. k-fold CV 평균은 “진짜 성능”의 공정한(무편향) 추정치?

- “한 모델 평가” 용도라면 같은 분포의 새 데이터 기대 오차에 대한 합리적 추정치.
- “여러  $\lambda$  후보 중 최솟값을 고르는” 용도로 쓰면 그 최솟값은 평균적으로 낮게(낙관적) — 정직한 숫자가 필요하면 선택에 쓰지 않은 test(6.3)나 nested CV를 쓴다.

## 확인 문제 6.2

- 1 샘플 10개(인덱스 0~9)를  $k = 5$ 로 `k_fold_split`. `fold_idx=2`일 때 `val`에 들어가는 인덱스를 쓰고, 5개 fold를 모두 돌렸을 때 각 인덱스가 `val`에 총 몇 번 등장하는지 확인.
- 2 평균모델 예( $y = 1, \dots, 10$ , 순서 2개씩 5등분)에서 fold 0과 4의 fold별 MSE를 각각 계산하고 5-fold 평균을 구해, 정직한 8.25와 CV 12.75의 차이를 한 문장으로 설명.
- 3 아래  $\lambda$  5개에 대한 5-fold 평균 MSE: (가) 최소점은? (나) 최소점  $\lambda = 1.0$ 의 fold별 MSE = [4.2, 4.4, 4.6, 4.8, 5.0]일 때 1SE 규칙(최소점 평균 + fold 표준편차 1개 이내인 후보 중 가장 큰  $\lambda$ )으로 고를  $\lambda$ 는?

[ $\lambda$  : 0.1 0.3 1.0 3.0 10.0] / [MSE : 5.00 4.80 **4.60** 4.70 5.20]

- 4  $\lambda$  후보 20개 CV의 최소점(MSE 3.12)을 “배포 모델의 기대 성능”으로 보고했다. 이 주장이 왜 낙관 편향이 있는지 § 최적화 편향의 구조로 설명하고, 정직한 추정치를 얻는 두 방법(하나는 데이터들, 하나는 계산을 더 요구)을 쓰라.

## 6.2 정리

이 절은 두 가지를 손에 넣었다.

- ① **절차** — k-fold CV로 전체를 “한 번씩은 검증에도” 쓰면서, 학습 손실이 아니라 검증 손실이 가장 작은  $\lambda$ 를 고른다: U자형 곡선은 6.1의 편향-분산이 실제 숫자로 그려진 것이고, 1SE 규칙은 “최소점” 단일 수치에 대한 건강한 불신을 **제도화**한 것.
- ② **경고** — “여러 후보 중 최솟값을 고르는” 행위 자체가 또 하나의 선택이므로, CV 곡선의 최소점은 곧바로 “최종 성능”이 될 수 없다.

**낙관적 편향을 격리하는 방법 — 선택에 쓰지 않은 test, 또는 nested CV — 은 6.3의 train/val/test 분리 원리가 이 절과 맞닿는 지점.**

---

## 6.3 train/val/test 분리 원칙 실전

---

## 왜 3분할이 필요한가: 배우다와 재다를 나누어야

머신러닝 파이프라인은 두 가지 완전히 다른 일을 동시에 한다:

**배우는 일** train 데이터로 모델의 파라미터를  
정하는 것.

**재는 일** 그 모델이 보지 못한 데이터에서 얼마나  
잘하는지 평가.

### 이 둘을 같은 데이터로 하면

모델이 “재는 데이터”까지 보고 만들어졌으므로, 그 점수는 “학습을 얼마나 잘했는가”가 아니라  
“기억을 얼마나 잘했는가”를 재게 된다. 4.1의 과적합(“train에서 완벽, test에서 참사”)과 **정확히  
같은 구조.**

## 역할 분리: 각 조각이 딱 한 역할만

- **train**은 배운다 — 학습 데이터로 파라미터( $w$ )를 정.
- **val**은 모델을 고른다 — 하이퍼파라미터 ( $\lambda$ , kNN의  $k$ , 트리 깊이 등)를 정.
- **test**는 딱 한 번만 최종 점수를 준다.

### 한 줄 압축

**“정보의 흐름 방향을 정한다”**. train은 val·test로, val은 최종 모델로 정보가 흐를 수 있지만, test에서는 어디로도 정보가 흘러나와서는 안 된다. test는 파이프라인의 마지막 관문 — 한 번 지나가면 돌아올 수 없다.

# 정보 흐름 다이어그램



## 허용된 흐름과 금지된 흐름

- **train** → **val / test**: 전처리 통계량(평균·표준편차, PCA 축)을 train으로만 fit하고 val·test에는 apply — 허용되는 유일한 교차.
- **train** → **모델**:  $w$ 를 fit
- **test** → (**val / train / 모델**): 모두 금지

## 3분할이 실제로 깨지는 흔한 실수 세 가지

- ① **테스트로 여러 모델을 비교**해 가장 잘 나온 걸 고르는 것 (“살짝 훑쳐보기”).
- ② **검증 데이터 없이** 테스트 데이터로 하이퍼파라미터까지 같이 튜닝(검증과 테스트를 사실상 섞어버림).
- ③ 데이터를 나누기 **전에** 정규화(스케일링) 같은 전처리 통계량(평균, 표준편차)을 **전체 데이터**로 계산 — test 정보가 전처리 단계에서 이미 학습에 스며들(**data leakage**).

### 더 좋은 질문

“이 코드에 leakage가 있는가?”보다: **“test(또는 val)의 정보가 학습이나 선택 단계에 닿았는가?”**

아래 §에서 이 질문을 **숫자로** 답할 수 있는 도구를 하나씩 만든다.

## 손으로 한 번: 누수를 숫자로

| 기준             | 평균  | 표준편차  |
|----------------|-----|-------|
| 전체 10개(누수)     | 5.5 | 2.872 |
| train 7개만(올바름) | 4.0 | 2.000 |

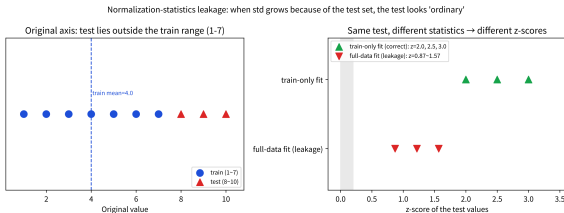
test = [8, 9, 10]를 각각의 통계량으로 표준화:

전체(누수) : [0.870, 1.219, 1.567]      train(올바름) : [2.0, 2.5, 3.0]

- 전체 데이터로 미리 계산한 평균·std를 쓰면 test 값이 실제보다 훨씬 “평범(z값 작음)”해 보인다 — std 자체가 test의 큰 값들 때문에 이미 커져 있기 때문(2.872 vs 2.0).

# 배포 시나리오: 미래를 볼 수 없다

- train 통계량으로 표준화



## 전체 fit 시 test z가 평범해짐

### 왜 std가 test 때문에 커지는가

전체 10개의 std(2.872)가 train 7개(2.0)보다 큰 것은, test의 큰 값들 (8,9,10)이 분산 계산에 이미 들어와 전체 std를 끌어올린 것.

재고 싶은 것(test)이 재는 자(통계량)를 미리 바꿔놓은 것 = 누수의 핵심.

# scikit-learn의 표준 구현

```
# 올바른 순서 : 으로부터 train fit, val/에는 test 만 transform
scaler = StandardScaler().fit(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

.fit\_transform(X\_all)을 val·test에 쓰면 “누수” 결과와 동일한 위장이 일어난다.

## “살짝 훑쳐보기”의 통계: 선택 편향

- 후보  $m$ 개 중 **최고 점**을 고르는 행위 =  $m$ 번의 잡음 뽑기 중 max를 고르는 것과 같음.
- $m$ 개 독립 정규분포의 max는 평균(=0, 진짜 성능)보다 **항상 위쪽으로 치우침** —  $m$ 이 커질수록 max는 더 위로.



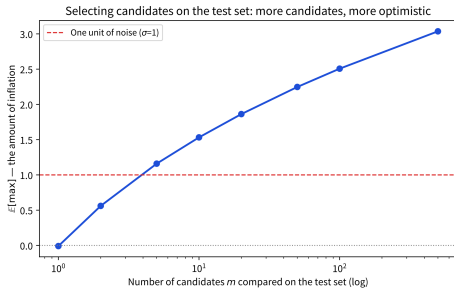
## Leaderboard 10.05% Display top 20 leaders.

| Rank                         | Team Name                                 | Best Score | % Improvement | Last Submit Time    |
|------------------------------|-------------------------------------------|------------|---------------|---------------------|
| 1                            | <a href="#">BellKor's Pragmatic Chaos</a> | 0.8558     | 10.05         | 2009-06-26 18:42:37 |
| Grand Prize - RMSE <= 0.8563 |                                           |            |               |                     |
| 2                            | <a href="#">PragmaticTheory</a>           | 0.8582     | 9.80          | 2009-06-25 22:15:51 |
| 3                            | <a href="#">BellKor in BigChaos</a>       | 0.8590     | 9.71          | 2009-05-13 08:14:09 |
| 4                            | <a href="#">Grand Prize Team</a>          | 0.8593     | 9.68          | 2009-06-12 08:20:24 |
| 5                            | <a href="#">Dace</a>                      | 0.8604     | 9.56          | 2009-04-22 05:57:03 |
| 6                            | <a href="#">BigChaos</a>                  | 0.8613     | 9.47          | 2009-06-23 23:06:52 |

Netflix Prize(2006~2009, 상금 \$1M) 공개 리더보드 — 참가자가 보는 quiz 세트 순위와 최종 판정에 쓰인 test 세트는 서로 분리되어 있었다(“살짝 훑쳐보고 고치기”를 막기 위해)

## $\mathbb{E}[\max]$ 를 숫자로

| $m$                | 1           | 2    | 5    | 10   | 20   | 50   | 100  | 500  |
|--------------------|-------------|------|------|------|------|------|------|------|
| $\mathbb{E}[\max]$ | $\approx 0$ | 0.56 | 1.16 | 1.53 | 1.86 | 2.25 | 2.51 | 3.04 |



후보 수  $m$ 에 따른  $\mathbb{E}[\max]$ 와 잡음 비교

- $m = 50$ 이면 잡음의 2배 이상 부풀어 오름

val에 걸리는 편향은 정직하다

$m$ 이 크면 낙관적 상한에 가까움

## 자주 하는 실수: 시계열을 무작위로 나누기

추세(매일 +1)가 있는 일일 데이터 365일, kNN:

| 분할 방식                | test RMSE |
|----------------------|-----------|
| 무작위 분할               | 0.81      |
| 시간순 분할(뒤 20% = test) | 44.1      |

- 무작위 분할은 미래 값을 과거로 예측
- 시간순 분할은 외삽으로 RMSE 급등

### 교훈

이 **44.1**이 실제 일반화 성능에 가까운 값. 시계열은 무작위가 아니라 **시간순으로** 나눠야 하고, 더 근본적으로 **“실전에서 겹치지 않을 것”을 같은 조각에 두지 말라**. `train_test_split(shuffle=False)` / `TimeSeriesSplit`. 그룹(환자·기기·지점): `GroupKFold`.

## 실습: diabetes 3분할 파이프라인 전체

```
X, y = load_diabetes().data, load_diabetes().target # 건442, 개10 특징
# 1) 단계2 분할3: 를40% (val+test)로, 그를 40% 절반씩
Xtr, Xtemp, ytr, ytemp = train_test_split(X, y, test_size=0.4, random_state=0)
Xva, Xte, yva, yte = train_test_split(Xtemp, ytemp, test_size=0.5, random_state=0)
# 2) val 로만RMSE lambda 고르기선택 ( 편향은에서만 val)
lams = [0.1, 1.0, 10.0, 100.0, 1000.0]
val_rmse = {lam: np.sqrt(mean_squared_error(
    yva, Ridge(alpha=lam).fit(Xtr, ytr).predict(Xva))) for lam in lams}
best = min(val_rmse, key=val_rmse.get)
# 3) 그로 lambda 만train fit, test RMSE 한번만
final = np.sqrt(mean_squared_error(
    yte, Ridge(alpha=best).fit(Xtr, ytr).predict(Xte)))
```

(4) 대조로 (잘못) test로  $\lambda$ 를 고르면 어떤 차이가 나는지 본다.

## 실행 결과: 핵심은 숫자보다 절차

크기: train=265 val=88 test=89

val RMSE per lambda: {0.1: 58.07, 1.0: 61.75, 10.0: 73.04,  
100.0: 77.49, 1000.0: 78.07}

val로 고른 lambda = 0.1 -> 최종 test RMSE (한 번만) = 53.79  
(잘못) test로 고르면 lambda = 1.0

### 핵심

val로 고른  $\lambda$ 와 test로 고른  $\lambda$ 는 (이 작은 데이터에서) 같을 수도, 다를 수도 있다 — 위 실행에선 0.1 vs 1.0으로 다름.

우연히 같아도 원칙적으로 test를 선택에 썼으므로 그 test 수치는 선택편향의  $\mathbb{E}[\max]$ 를 먹고 있는 **부풀려진 추정치**.

**같은 모델·같은 코드인데도 “ $\lambda$ 를 어디서 고르느냐”만 달라져 최종 test 점의 신뢰성이 달라짐 — 3분할은 “코드가 아니라 절차”의 문제.**

## 2단계 3분할은 정말 겹치지 않는가

샘플 100개(0~99)를 “앞 60=train, 뒤 40=temp” 1차 분할 후 temp(60~99)를 “앞

|                        | 조각    | 인덱스 범위 | 개수 |
|------------------------|-------|--------|----|
| 20=val, 뒤 20=test” 2차: | train | 0~59   | 60 |
|                        | val   | 60~79  | 20 |
|                        | test  | 80~99  | 20 |

- 교집합이 빈집합인지

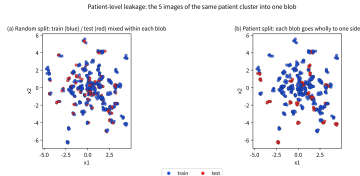
2단계 분할은 3분할 원칙 위반 아님

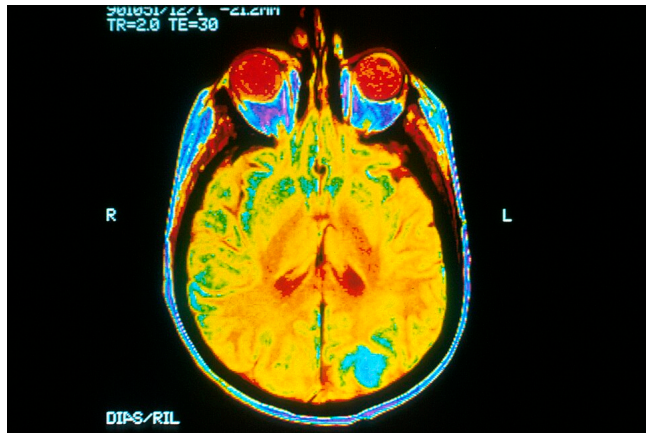
## 유명한 사례: 환자 단위 누수

- 논문에 보고된 정확도가 실제 새로운 환자에게 적용했을 때보다 **부풀려짐**.

| 분할 방식          | test RMSE |
|----------------|-----------|
|                |           |
| 무작위 분할(이미지 단위) | 1.04      |
| 환자(그룹) 분할      | 3.65      |

환자 100명, 각 5장 스캔, kNN:





뇌 MRI 스캔 한 장 — 같은 환자의 여러 장은 서로 매우 비슷해, 무작위로 train/test에 흩어지면 “새 환자”가 아니라 “본 적 있는 환자”를 다시 알아보는 누수가 생긴다

# 환자를 “외운다” — 왜 1.0이 “성취”가 될 수 없는가

- 무작위 분할 RMSE( $\approx 1.0$ )가 환자별 값의 표준편차(2.0)보다 훨씬 낮음.
- 새로운 환자를 예측하는 것은 이론적으로  $\sigma\sqrt{2} \approx 2.8$  아래로 내려갈 수 없는데 1.0을 “성취”했다.
- 이걸 모델이 관계를 배운 것이 아니라 환자를 외운 것.

```
from sklearn.model_selection import GroupKFold
groups = np.repeat(np.arange(100), 5) # 환자100 x 장5, 그룹키환자=
gkf = GroupKFold(n_splits=5)
for tr_idx, te_idx in gkf.split(X, y, groups=groups):
    pass # 같은환자의장은 5 항상 train 또는 test 중하나로만
```

## 자주 묻는 질문 (1)

### Q. 데이터가 너무 적어 3분할하면 각 조각이 너무 작아지는데?

- test를 따로 떼는 대신, 6.2의 교차검증으로 “학습+검증”을 반복하고, 정말 최종 성능이 필요할 때만 별도로 확보한 **작은 test로 딱 한 번** 확인하는 절충안.
- 극도로 적으면 LOOCV까지 고려. 그래도 **test를 완전히 없애고 검증 성능만으로 최종 성능을 주장하는 것은 피해야** 한다 — 검증 성능은 이미 하이퍼파라미터 선택에 “간접적으로” 쓰였기 때문.
- **몇 대 몇?** 정해진 정답 없음 — 60/20/20이나 70/15/15가 흔한 시작점. 아주 많으면(수백만) val·test가 각각 1~2%만 되어도 안정적.

### Q. 교차검증(6.2)을 쓰면 test는 굳이 필요 없지 않나요?

- 보통은 그렇지만, “정확히 한 번”의 최종 성능이 필요한 (논문 제출, 배포 전 검증) 상황에선 필요.
- k-fold CV는 val 역할(하이퍼파라미터 고르기), test는 최종 판정(아무 선택 없이 처음 보는 데이터로 검증). CV만 있으면 “고르는 과정”만 있고 “판정”이 없다.
- 특히 **CV를 여러 번( $\lambda$ 마다, 시드마다) 돌리면서** test 점수를 보고 결정 내리면, test가 다시 선택에 쓰임.

## 자주 하는 실수: 전처리 fit 시점과 “살짝만 조정”

- 그 반대 — 파이프라인으로 묶는 것이 leak을 막는 정석.
- Pipeline fit 시 train만 사용
- 전처리 fit은 항상 분할 이후, train 조각에만.
- test 재조정 시 편향 발생

## 확인 문제 6.3

- 1 테스트 정보가 다른 단계로 흘러나가는 것에 모두 체크: (a) val RMSE로  $\lambda$  고르기 (b) test RMSE로  $\lambda$  고르기 (c) `StandardScaler().fit(X_all)` 후 train/val/test에 transform (d) train으로 fit한 스케일러로 test transform (e) test에서 제일 잘 맞는 모델을 보고, “더 잘 맞추려고” train에 test를 추가로 넣기.
- 2 모델 후보 10개  $\times$   $\lambda$  후보 10개 = 100개 조합을 테스트로 평가해 최고 조합 고름. 잡음  $\sigma = 0.5$ 라면, “고른 조합의 test 점”은 진짜 성능보다 **평균적으로** 얼마나 높을까? (힌트:  $\mathbb{E}[\max]$ 를  $\sigma$ 로 곱해라.)
- 3 아래 2단계 3분할 중 원칙 위반: (가) 80/20  $\rightarrow$  val/test 10/10, (나) 80/20  $\rightarrow$  test에서 제일 좋은  $\lambda$  골라 train/val에 다시 적용, (다) train으로 fit한 PCA로 val/test 투영.
- 4 “데이터 100개뿐이라 3분할하면 test가 20개밖에 안 돼서 val을 빼고 80/20만 쓰고, test로  $\lambda$ 를 고르되 최종 수치는 그 test의 5-fold 평균으로 보고하자”는 제안을 평가 — 어떤 점이 원칙을 지키고, 어떤 점이 위반하는가?

## 6.3 정리

- 정규화는 “모델을 덜 유연하게 만들어 분산을 줄인다”는 4.1의 편향-분산 원리를, 손실함수에 항 하나를 더하는 것만으로 실제로 조절 가능하게 만든 도구 — 그리고 그 조절 손잡이( $\lambda$ ) 자체는 교차검증이라는 또 다른 절차로 정해야 한다.
- 그 교차검증/분리 절차 자체가 이 절의 주제: **train으로  $w$ , val로  $\lambda$ , test로 딱 한 번의 최종 점수.**
- 이 세 역할을 한 조각이 두 가지로 겹치면(특히 test가 재는 것과 고르는 것을 겹치면), 선택편향이 보여준 것처럼 보고된 숫자는 진짜 성능이 아니라 **잡음의 max**가 되어버린다.

3분할은 “데이터를 셋으로 쪼갬다”는 기계적 절차가 아니라, **정보의 흐름에 방벽을 세우는 일.**

## 이번 주차 정리

- ❶ **6.1 페널티의 모양이 결과를 바꾼다** —  $\lambda$ 를 올리면 분산을 편향과 바꿈. L2는 매끄러운 축소, L1은 soft-thresholding으로 **정확히 0**에 스냅시켜 특징 선택을 무상으로(원 vs 마름모). bias는 정규화에서 제외.
- ❷ **6.2 학습 손실이 아니라 검증 손실로  $\lambda$ 를 고른다** — k-fold CV로 U자형 곡선을 그리며, 1SE 규칙으로 “최소점” 단일 수치에 대한 불신을 제도화. “여러 후보 중 최솟값을 고르는” 행위 자체가 선택 → **최적화 편향**, nested CV로 격리.
- ❸ **6.3 train/val/test 3분할** — train은 배운다, val은 고르고, test는 **딱 한 번**. “정보 흐름의 역주행”(leakage)을 숫자로: 선택 편향( $\mathbb{E}[\max]$ ), 전처리 통계량 누수, 시계열·환자 단위 분할.

### 다음 주 — Week 7. 트리 기반 모델

정규화·교차검증·3분할은 다음 장의 트리·양상블(GBDT) 튜닝이 올라설 **초석**. 트리의 복잡도 손잡이는 가중치 페널티가 아니라 “트리 깊이”, “분할 횟수”가 되지만, “수많은 후보 중 어떤 설정이 최적인가?”라는 질문은 **똑같이** 남는다 — 그 답을 만드는 틀(검증, 교차검증, train/val/test)이 바로 이 장에서 지은 것이다.